Politecnico di Torino

Image Processing and Computer Vision

Object detection system for amateur volleyball matches

Student ID: s345370 ***Author:*** Niccolò Malgeri

Student ID: s345156 ***Author:*** Luca Genca

Student ID: s333964 ***Author:*** Gabriele Piergiovanni

Academic year: 2025–2026

Contents

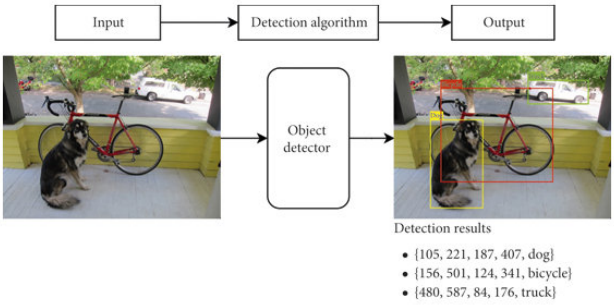


Figure 1: Example of an object detection task

Figure 2: Example of an amateur volleyball court

1 Introduction

In Computer Vision, **Object Detection** is the task of detecting the presence of certain objects in an image. In particular, an object detecion algorithm is able to find the position of objects by placing a bounding box (described by its coordinates, width and height) around them and to recognize the class they belong to (**Fig.** 1). This project's goal is to apply object detection in the context of amateur volleyball matches. Given a video stream coming from a camera filming a volleyball match, the algorithms used should perform the following:

- detect in real time the players by distinguishing them from the public
- detect in real time the ball
- detect in real time the lines of the court (an example of it in **Fig.** 2)
- zoom on the ball when it's near the players

In particular, we evaluate and compare three versions of the detection system:

- one created with the aim of being robust to different test settings
- one created with the aim of working only on a specific test setting, under specific lighting conditions
- one created with the aim of working only on a specific test setting, but being also robust to different lighting conditions

The object detection "protagonist" for this project is ***YOLO*** (***You Look Only Once***, one of the most popular neural-network-based models for this type of task. During the years several versions have been released, but for all the experiments we decide to focus only on *version 8* (*YoloV8x* models); in particular, ***YoloV8n*** (*nano*, light version with less weights) and ***YoloV8m*** (*medium*, bigger version with more weights) are considered.

2 Dataset creation

The training images come from official volleyball matches on YouTube (1). In particular, we extract evenly spaced frames from several videos, obtaining 438 images that we split in 311 images for the training set and 127 images for the validation set. Then, via *RoboFlow*, an online platform for annotating images, we add ground truth bounding boxes to each volleyball court and ball (separately). The test set is created by extracting 20 random frames from 2 test videos coming from a volleyball match held in Turin in Summer 2025.

3 Experimental setup

3.1 Images Preprocessing

For the court detection, three types of preprocessing are considered:

1. **Lines Detection:** the input image is converted to grayscale, and edges are detected using the Canny edge detector. Straight lines are then extracted using the probabilistic Hough transform (`HoughLinesP`) and drawn on the image. This emphasizes linear features such as lane markings
2. **Color Thresholding:** the image is converted to HSV color space, and masks are created for white and orange color ranges. These masks are combined and applied to the image to preserve only regions of interest, again enhancing features like lane markings while suppressing irrelevant background
3. **Combined Thresholding and Lines:** this method first applies the white and orange color thresholding to isolate relevant regions. The masked image is then converted to grayscale, edges are detected, and lines are extracted with the probabilistic Hough transform. Detected lines are then drawn on the masked image

Results are reported only for the **Combined Thresholding and Lines** preprocessing as it provides the best performance.

3.2 Court detection

To detect the court a YoloV8n model is used. Different parameters and training/testing configurations are selected based on the preprocessing and augmentation considered. The following tables summarize the choices taken:

Configuration 1: Preprocessing without test frames

Parameter	**Value**
Model	YOLOv8n
Image size	640
Epochs	100
Batch size	10
Learning rate (lr_0)	0.0001
Weight decay	0.0003
Classification weight (cls)	0.5
Data augmentation	No
Confidence threshold	0.3
Preprocessing	Combined color thresholding and line detection
Test frames used	No

Table 1: Training/testing configuration for court detection with preprocessing, without test frames

Configuration 2: Preprocessing with test frames

Parameter	Value
Model	YOLOv8n
Image size	640
Epochs	100
Batch size	10
Learning rate (lr_0)	0.0001
Weight decay	0.0003
Classification weight (cls)	0.5
Data augmentation	No
Confidence threshold	0.3
Preprocessing	Combined color thresholding and line detection
Test frames used	Yes

Table 2: Training/testing configuration for court detection with preprocessing and test frames

Configuration 3-4: No preprocessing, with test frames

Parameter	Value
Model	YOLOv8n
Image size	1056
Epochs	100
Batch size	10
Learning rate (lr_0)	0.01
Weight decay	0.0005
Classification weight (cls)	0.5
Data augmentation	Yes/No
Confidence threshold	0.3
Preprocessing	None
Test frames used	Yes

Table 3: Training/testing configuration for court detection without preprocessing, with test frames (with/without augmentation)

Parameter	Range
Hue	$\pm 15°$
Saturation	$\pm 25\%$
Brightness	$\pm 18\%$
Exposure (Gamma)	$\pm 8\%$

Table 4: Augmentation parameters applied to the training set

For the data augmentation part, hue, saturation, brightness and exposure are considered (details in **Table 4)**. These modifications are chosen to simulate different lighting conditions, which could be present in the test environment.

3.3 Players detection

For players detection, a YOLOv8m model pre-trained on the COCO dataset is chosen, without any additional training. Similarly to court detection configurations, during inference, a confidence threshold of 0.3 is applied to filter the detections.

3.4 Ball detection

Ball detection was performed using YOLOv8s, trained on YouTube clips annotated via Roboflow as already stated. There were two main issues that occured by working with object such as volleyball balls, the first one being their size in comparison to the whole image and the second one being the fact that the ball gets thrown fast, and that causes motion blur, working just with the images on the dataset gave the result of performing well on clear frames but failing when the ball appeared smeared due to high velocity. For the first problem we increased the size of the images that the model would be trained with, for the second problem we worked with a dataset augumentation approach that we'll discuss below.

3.4.1 Motion Blur Augmentation

The dataset augumentation script that we wrote expands the original dataset by 50% by introducing synthetic motion blur. The algorithm applies a directional filter (linear kernel) with randomized degrees and angles to mimic the shutter speed artifacts observed in match footage and, in addition, to generalize across different arena lightings, random photometric noise (brightness and contrast variations) is injected into the blurred samples. Also, the original ground truth labels are preserved, allowing the network to learn feature correlation between the blurred visual patterns and the object's position. Working this way there is a noticeable difference between the behaviour of the model, especially in situations where the ball is moving at high speed.

Train configuration for the ball detection

Parameter	**Value**
Model	YOLOv8s
Image size	1280
Epochs	100
Batch size	4
Learning rate (lr_0)	0.0001
Weight decay	0.0003
Classification weight (cls)	0.5
Confidence threshold	0.3
Preprocessing	Synthetic blur

Table 5: Training/testing configuration for ball detection with preprocessing

(a) Example of ball frame (b) Example of ball frame after the synthetic blur

Figure 3: Comparison between the different images in the enhanced dataset, both zoomed

3.5 Experiments

Regarding the experiments, each configuration described in **Section 3.2** is applied to two main test cases:

- video sequences of the volleyball match in Turin
- static frames contained in the test split, extracted from the same volleyball match video

3.5.1 Performance metrics

To assess the quality of the detections, three main metrics are considered:

- **precision**: is the number of correctly predicted detections over all the predicted detections:

$$P = \frac{TP}{TP + FP} \tag{1}$$

- **recall**: is the number of correctly predicted detections over all the ground truth detections (i.e.: all the existing bounding boxes):

$$R = \frac{TP}{TP + FN} \tag{2}$$

- **mean average precision (mAP)**: is the mean of the **Average Precision** (**AP**) values across all detection classes (in our case it's just the court class). The AP corresponds to the area under the *Precision-Recall curve*:

$$AP = \int_0^1 p(r)\, dr \tag{3}$$

  For reporting the mAP metric, we consider two common conventions:

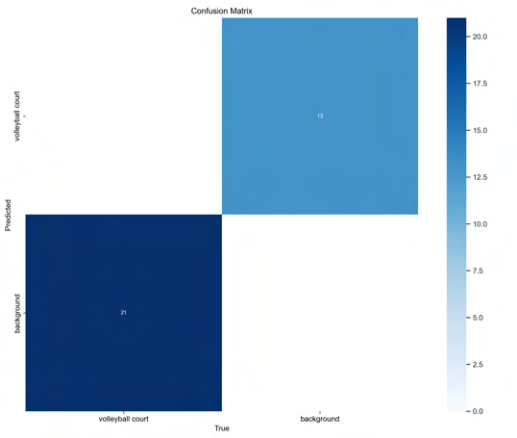
  - **mAP@50**: the mAP computed at a single *Intersection over Union* (IoU) threshold of 0.50. A prediction is considered a True Positive if it overlaps at least 50% with the ground truth.
  - **mAP@50-95**: the average of the mAP computed at 10 different IoU thresholds, from 0.50 to 0.95 with steps of 0.05

3.5.2 Augmentation in the test set

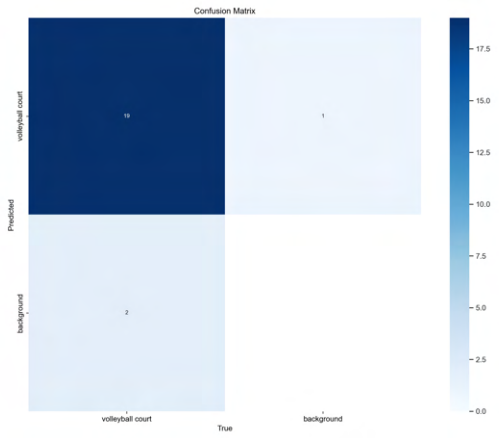
To check the model's robustness in the test scenario, we apply augmentations similar to the ones described in Table 4 also for the test frames. In particular, the parameters considered are the same (hue, saturation, brightness, gamma correction), but for each of them the variation range is slightly increased.

4 Test results

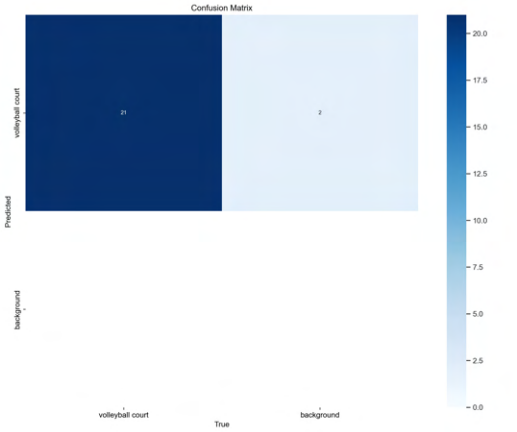
4.1 Confusion Matrix Comparison



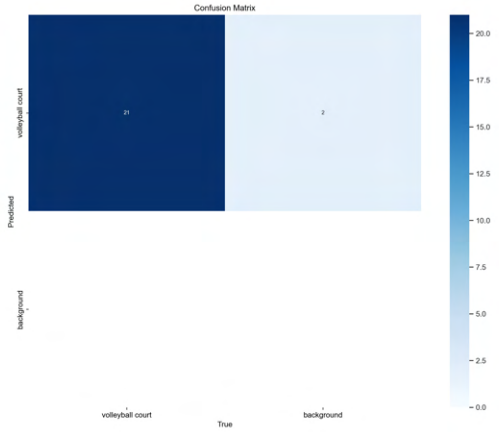
(a) Preprocessing w/o test frames



(b) Preprocessing w/ test frames



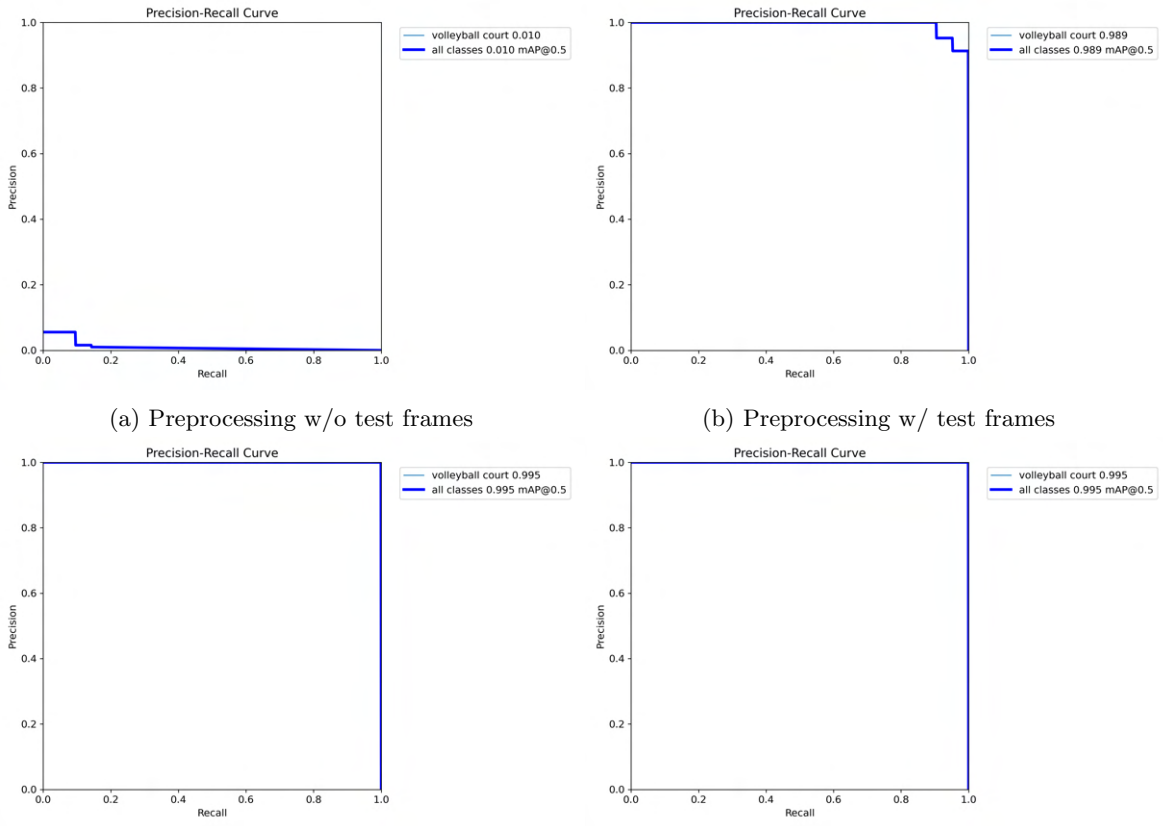
(c) No Preprocessing w/ test frames



(d) No Preprocessing w/ test frames & aug

Figure 4: Confusion Matrices across all four configurations

4.2 Precision-Recall (PR) Curve Comparison



(a) Preprocessing w/o test frames

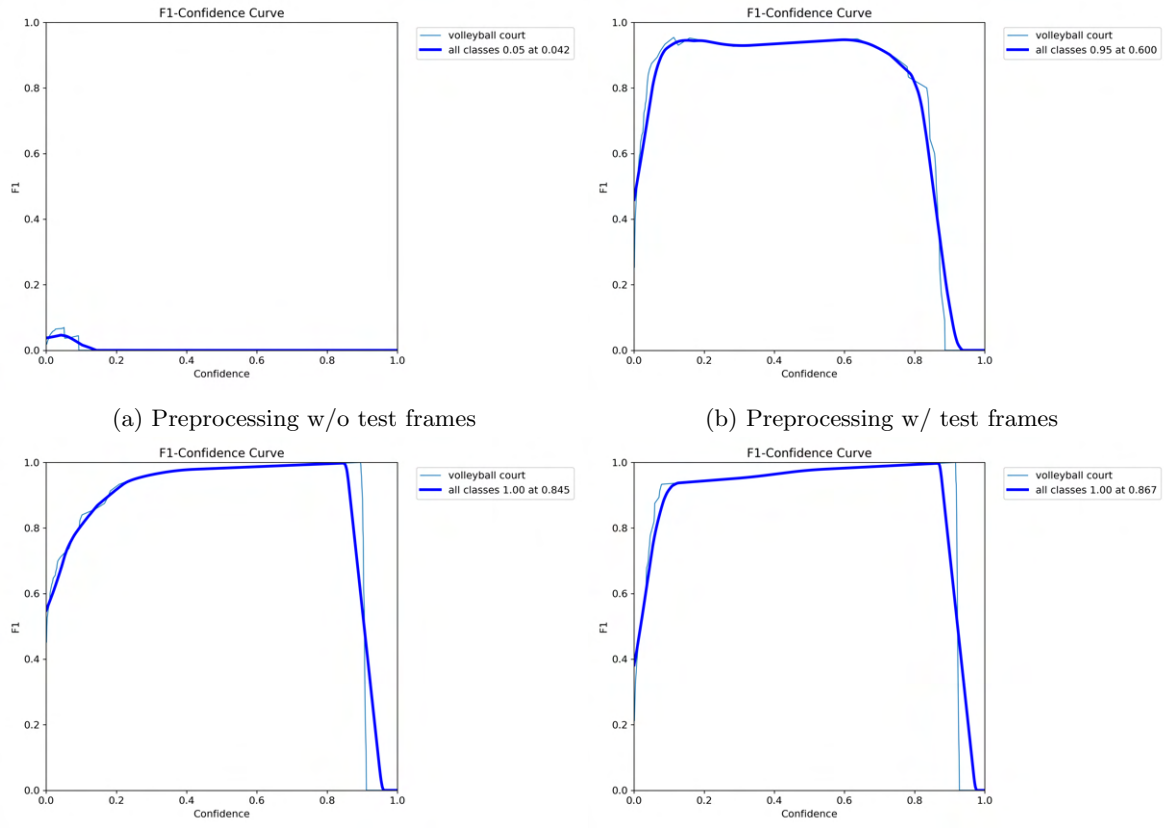
(b) Preprocessing w/ test frames

(c) No Preprocessing w/ test frames

(d) No Preprocessing w/ test frames & aug

Figure 5: PR Curves across all four configurations

4.3 F1 Score Comparison



(a) Preprocessing w/o test frames

(b) Preprocessing w/ test frames

(c) No Preprocessing w/ test frames

(d) No Preprocessing w/ test frames & aug

Figure 6: F1 Scores across all four configurations

4.4 Results on the test video

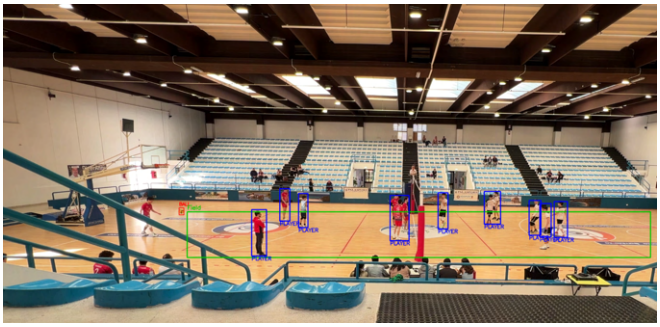


Figure 7: Performance of the full detection system (ball+court+players) on the test video sequence

Fig. 7 shows the performance of the full detection system (ball, court and players detection) on the test video sequence. Even though at higher confidence thresholds the ball detection starts failing (for the tests we find reasonable to use a confidence value of 0.3), the pipeline achieves good results. The results on the static frames of the test set are summarized in **Tables 6** and **7**.

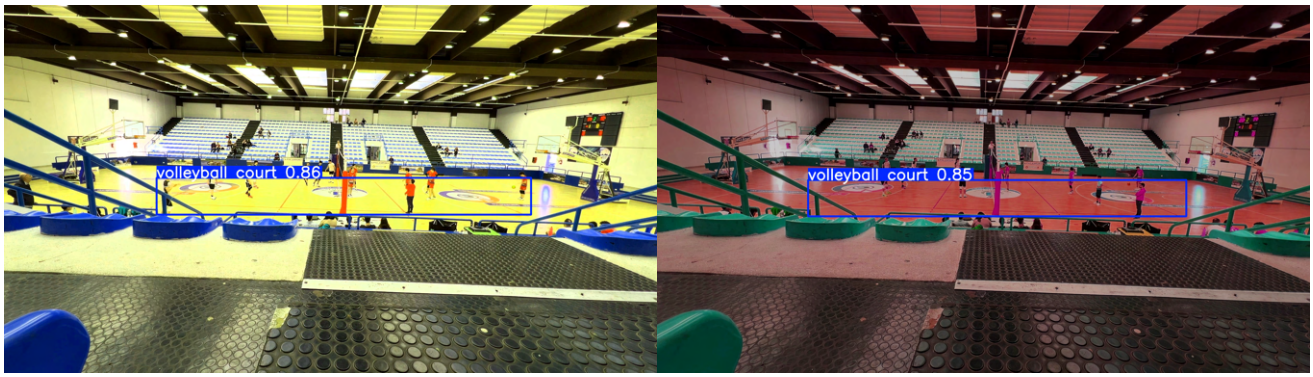
Training Configuration	Precision	Recall	mAP@50	mAP@50–95
No preprocessing with test frames and augments	0.9915	**1.0000**	**0.9950**	0.7027
No preprocessing with test frames	0.9961	**1.0000**	**0.9950**	0.6774
Preprocessing with test frames	**0.9973**	**1.0000**	**0.9950**	**0.7758**
Preprocessing without test frames	0.0116	0.2381	0.0076	0.0015

Table 6: Comparison of model performance metrics on base test set for different training strategies

Training Configuration	Precision	Recall	mAP@50	mAP@50–95
No preprocessing with test frames and augments	**0.9956**	**1.0000**	0.9950	0.6965
No preprocessing with test frames	**0.9956**	**1.0000**	0.9950	0.6908
Preprocessing with test frames	0.9948	0.9048	**0.9891**	**0.7505**
Preprocessing without test frames	0.0510	0.0952	0.0103	0.0019

Table 7: Comparison of model performance metrics on augmented test set for different training strategies

5 Analysis of results



(a) first frame (b) second frame

Figure 8: Results on two test video frames (high HSV distortion)

In the augmented setup, as reported in **Table 7**, the model trained with preprocessing and test frames achieves the highest values in all the metrics considered. However, in the non-augmented setup (**Table 6**) it exhibits considerably lower recall compared to the no-preprocessing models, revealing its lack of robustness to changes in lighting; this is likely due to thresholding failures when test images colors differ significantly from those seen during training. Specifically, lighting alterations cause highly deviated HSV levels, leading the thresholding process to highlight unexpected regions (examples of highly modified images are shown in **Fig 8**). Unsurprisingly, the model trained without frames from the test video yields the worst performance in both scenarios. This is attributed to the significant domain shift between the test and training environments.

6 Conclusions

Tables 6 and **7** prove the success of the YOLO model across nearly all tested configurations. Assuming access to the test environment for training data, both the preprocessing and non-preprocessing versions are viable strategies, but with some trade-offs: the preprocessing model is preferable for maximizing mAP, whereas the model without preprocessing can be preferred when prioritizing robustness to environmental changes at inference time. Since only YOLOv8n and YOLOv8m are used for these experiments, other YOLO versions and sizes could be considered as well.

7 References

[1] https://www.youtube.com/c/volleyballworld